

10 useful NumPy functions

1. np.tile()

- **What it does:** Constructs a new array by repeating an input array the number of times given by reps.
- **Syntax:** np.tile(A, reps)
- **Example:**

```
import numpy as np  
arr = np.array([1, 2])  
result = np.tile(arr, 3)  
print(result)
```

2. np.repeat()

- **What it does:** Repeats elements of an array individually a specified number of times.
- **Syntax:** np.repeat(a, repeats, axis=None)
- **Example:**

```
import numpy as np  
arr = np.array([1, 2])  
result = np.repeat(arr, 3)  
print(result)
```

3. np.meshgrid()

- **What it does:** Returns coordinate matrices from coordinate vectors, which is useful for evaluating functions on a 2D grid.
- **Syntax:** `np.meshgrid(*xi, indexing='ij')`
- **Example:**

```
import numpy as np  
  
x = np.array([1, 2, 3])  
y = np.array([4, 5])  
  
X, Y = np.meshgrid(x, y)  
  
print("X", X)  
print("Y", Y)
```

4. np.select()

- **What it does:** Returns an array drawn from elements in a choice list, depending on specified conditions.
- **Syntax:** `np.select(condlist, choicelist, default=0)`
- **Example:**

```
import numpy as np  
  
x = np.array([1, 5, 10, 15])  
  
conditions = [x < 5, (x >= 5) & (x < 12), x >= 12]
```

```
choices = ['Low', 'Medium', 'High']
```

```
result = np.select(conditions, choices, default='Unknown')
```

```
print(result)
```

5. np.pad()

- **What it does:** Pads an array with values (zeros, constants, edge values, etc.) along specified axes.
- **Syntax:** `np.pad(array, pad_width, mode='constant', constant_values=0)`
- **Example:**

```
import numpy as np
```

```
arr = np.array([1, 2, 3])
```

```
padded = np.pad(arr, (2, 2), mode='constant', constant_values=0)
```

```
print(padded)
```

6. np.cumsum()

- **What it does:** Calculates the cumulative sum of array elements along a given axis.
- **Syntax:** `np.cumsum(a, axis=None)`
- **Example:**

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4])
```

```
result = np.cumsum(arr)
```

```
print(result)
```

7. np.squeeze()

- **What it does:** Removes single-dimensional entries (dimensions of size 1) from the shape of an array.
- **Syntax:** np.squeeze(a, axis=None)
- **Example:**

```
import numpy as np
```

```
arr = np.array([[[1, 2, 3]]]) # Shape: (1, 1, 3)
```

```
squeezed = np.squeeze(arr)
```

```
print("New shape:", squeezed.shape)
```

8. np.column_stack()

- **What it does:** Stacks 1D arrays as columns into a 2D array.
- **Syntax:** np.column_stack(tup)
- **Example:**

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
result = np.column_stack((a, b))
```

```
print(result)
```

9. np.isin()

- **What it does:** Calculates element in test_elements, returning a boolean array of the same shape as the input.
- **Syntax:** np.isin(element, test_elements)
- **Example:**

```
import numpy as np  
arr = np.array([10, 20, 30, 40, 50])  
targets = [20, 40, 60]  
mask = np.isin(arr, targets)  
print(mask)
```

10. np.apply_along_axis()

- **What it does:** Applies a custom function to 1D slices along the specified axis.
- **Syntax:** np.apply_along_axis(func1d, axis, arr, *args, **kwargs)
- **Example:**

Python

```
import numpy as np  
def diff_max_min(x):  
    return x.max() - x.min()
```

```
matrix = np.array([[10, 2, 3],  
                   [4, 25, 6]])  
result = np.apply_along_axis(diff_max_min, axis=1, arr=matrix)  
print(result)
```